

A). Project Overview (AgriPredict)

Project Name: AgriPredict

Type: Machine Learning Web Application

Backend: Python (Flask)

Frontend: HTML + JavaScript (Vite setup)

Database: SQLite

ML Model: XGBoost

Main Purpose

This project predicts **agricultural crop yield or output** based on different farming conditions such as:

- Crop type
- Soil type
- Region
- Other numerical features used in the ML model

It also includes **user authentication with OTP verification**.

So the system allows users to:

1. Create an account
2. Verify email using OTP
3. Log in
4. Enter agricultural data
5. Get ML-based predictions

B). Main Components of the Project

1. Backend (Flask)

Main file:

app.py

This is the **core of the application**.

Main responsibilities:

- Starts the Flask server
- Loads machine learning models
- Handles API requests
- Manages login / signup
- Returns predictions

Important libraries used:

- Flask
- numpy
- pandas
- pickle, flask_cors

2. Machine Learning Models

These files store the trained model and preprocessing objects:

```
encoders.pkl  
scaler.pkl  
xgb_model.pkl
```

What they do

encoders.pkl

- Converts categorical data into numeric form
- Example:
 - Wheat → 0
 - Rice → 1

scaler.pkl

- Normalizes numerical input data

xgb_model.pkl

- Trained **XGBoost machine learning model**
- Generates final predictions

Flow:

```
User Input  
↓  
Encoding  
↓  
Scaling  
↓  
XGBoost Model  
↓  
Prediction Output
```

3. Database

File:

```
agri_predict.db
```

This is a **SQLite database**.

Used for storing:

- User accounts
- Email
- Passwords
- Verification status

Database functions are in:

```
db_utils.py
```

Important functions:

- `init_db()` → create tables
 - `add_user()` → register user
 - `get_user_by_email()` → find user
 - `verify_user()` → email verification
 - `update_password()` → password reset
-

4. Email Service

File:

```
email_service.py
```

Purpose:

Handles **OTP email verification**.

Functions:

- `generate_otp()` → creates OTP
- `send_otp_email()` → sends verification code
- `send_reset_password_email()` → sends reset link

OTP is stored temporarily in memory:

```
otp_store = { email: otp }
```

5. Frontend

Frontend files include:

```
index.html  
templates/  
static/  
src/
```

index.html

Main webpage interface.

templates/

Used by Flask to render pages.

static/

Contains static assets like:

CSS, JavaScript, Images

src/

Frontend code (possibly built using **Vite + TypeScript**).

Supporting files:

```
package.json  
vite.config.ts  
tsconfig.json
```

These indicate a **modern frontend build system**.

6. Node Dependencies

```
node_modules/
```

Contains all frontend dependencies.

⚠ This folder should **NOT be pushed to GitHub**.

It will be recreated with:

```
npm install
```

7. Other Files

inspect_features.py

Used to inspect model input features.

cloudflared.exe

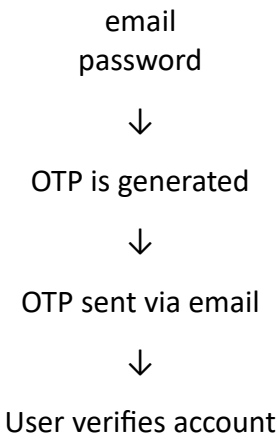
Tool used to expose **localhost to the internet** (for testing).

C). How the Whole System Works

Step-by-step workflow:

Step 1 — User Registration

User enters:



Step 2 — Login

User logs in using:

email
password

Session starts.

Step 3 — Input Data

User provides agricultural data:

Example:

Crop: Wheat
Soil: Black
Region: North
Other numeric features

Step 4 — Data Processing

Input goes through:

1. Encoder
2. Scaler
3. XGBoost model

Step 5 — Prediction

The model returns a predicted output.

Example:

Predicted Yield: 3.5 tons/hectare

D). Project Structure

```
AgriPredict
├── app.py                # Main Flask application
├── db_utils.py           # Database operations
├── email_service.py      # OTP email handling
├── agri_predict.db       # SQLite database
├── encoders.pkl          # ML encoder
├── scaler.pkl            # Data scaler
├── xgb_model.pkl         # XGBoost model
├── templates/           # HTML templates
├── static/              # CSS, JS, assets
├── src/                 # Frontend source
├── index.html
├── package.json
├── vite.config.ts
└── node_modules/        # Frontend dependencies
```